Лабораторная работа №4

дисциплина: Архитектура компьютеров

Савенкова Татьяна Александровна

Содержание

1 Цель работы 5

2 Задание 6

3 Теоретическое введение 7

4 Выполнение лабораторной работы 10

5 Выводы 15

Список литературы 16

Список иллюстраций

4.1 Установка необходимого ПО 10
4.2 Установка через pnpm 11
4.3 Создание репозитория 11
4.4 Конфигурация package.json 12
4.5 Отправление коммита на GitHub 12
4.6 Использование git-flow 13
4.7 Загрузка релиза на github 13
4.8 Работа с новой веткой и загрузка релиза 14

Список таблиц

1 Цель работы

Получение навыков правильной работы с репозиториями git.

2 Задание

- Выполнить работу для тестового репозитория.
- Преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits.

3 Теоретическое введение

Общая информация

Gitflow Workflow опубликована и популяризована Винсентом Дриссеном.

Gitflow Workflow предполагает выстраивание строгой модели ветвления с учётом выпуска про

Данная модель отлично подходит для организации рабочего процесса на основе релизов.

Работа по модели Gitflow включает создание отдельной ветки для исправлений ошибок в рабоч

Последовательность действий при работе по модели Gitflow:

Из ветки master создаётся ветка develop.

Из ветки develop создаётся ветка release.

Из ветки develop создаются ветки feature.

Когда работа над веткой feature завершена, она сливается с веткой develop.

Когда работа над веткой релиза release завершена, она сливается в ветки develop и maste

Если в master обнаружена проблема, из master создаётся ветка hotfix.

Когда работа над веткой исправления hotfix завершена, она сливается в ветки develop и m

Процесс работы с Gitflow

Основные ветки (master) и ветки разработки (develop)

Для фиксации истории проекта в рамках этого процесса вместо одной ветки master использ

При использовании библиотеки расширений git-flow нужно инициализировать структуру в с

```
git flow init
```

Для github параметр Version tag prefix следует установить в v.

После этого проверьте, на какой ветке Вы находитесь:

```
git branch
```

Функциональные ветки (feature)

Под каждую новую функцию должна быть отведена собственная ветка, которую можно отправл

Как правило, ветки feature создаются на основе последней ветки develop.

Создание функциональной ветки

Создадим новую функциональную ветку:

```
git flow feature start feature_branch
```

Далее работаем как обычно.

Окончание работы с функциональной веткой

По завершении работы над функцией следует объединить ветку feature_branch с develop

```
git flow feature finish feature_branch
```

Ветки выпуска (release)

Когда в ветке develop оказывается достаточно функций для выпуска, из ветки develop соз

допускается лишь отладка, создание документации и решение других задач. Когда подготовка

Благодаря тому, что для подготовки выпусков используется специальная ветка, одна коман

Создать новую ветку release можно с помощью следующей команды:

```
git flow release start 1.0.0
```

Для завершения работы на ветке release используются следующие команды:

```
git flow release finish 1.0.0
```

Ветки исправления (hotfix)

Ветки поддержки или ветки hotfix используются для быстрого внесения исправлений в рабо

Наличие специальной ветки для исправления ошибок позволяет команде решать проблемы, не

Ветку hotfix можно создать с помощью следующих команд:

```
git flow hotfix start hotfix_branch
```

По завершении работы ветка hotfix объединяется с master и develop:

```
git flow hotfix finish hotfix_branch
```

4 Выполнение лабораторной работы

Устанавливаю nodejs, пакетный менеджер для него pnpm и gitflow.(рис. 4.1).

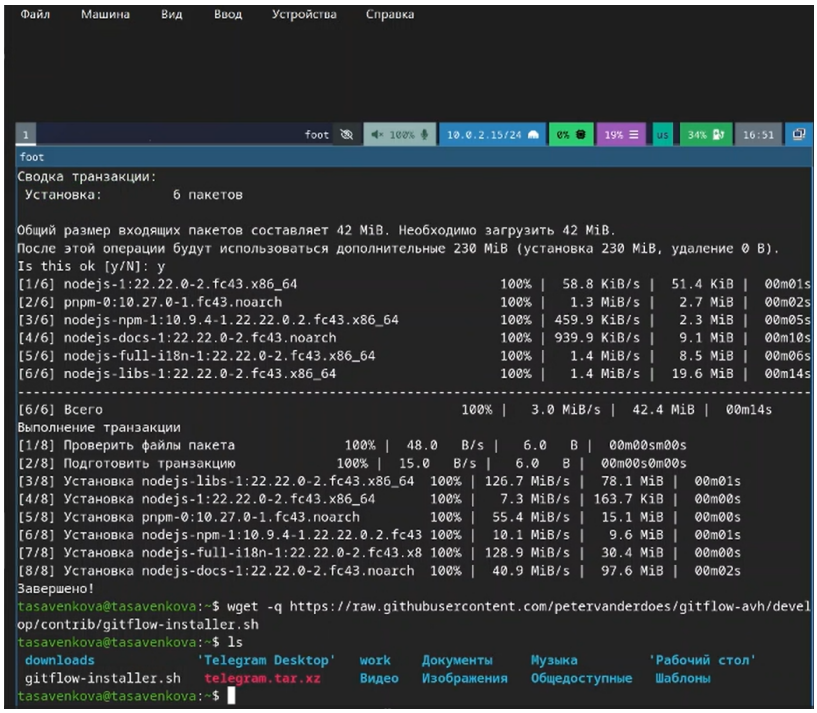

Рисунок 4.1: Установка необходимого ПО

Устаналиваю через pnpm commitizen и standard-changelog. (рис. 4.2)

```
Next configuration changes were made:
export PNPM_HOME="/home/tasavenkova/.local/share/pnpm"
case ":$PATH:" in
  *":$PNPM_HOME:"*) ;;
  *) export PATH="$PNPM_HOME:$PATH" ;;
esac

To start using pnpm, run:
source /home/tasavenkova/.bashrc
tasavenkova@tasavenkova:~$ source .bashrc
tasavenkova@tasavenkova:~$ pnpm add -g commitizen

    Update available! 10.27.0 → 10.30.3.
    Changelog: https://pnpm.io/v/10.30.3
      To update, run: pnpm self-update

 WARN  2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Packages: +150
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Progress: resolved 150, reused 0, downloaded 150, added 150, done

/home/tasavenkova/.local/share/pnpm/global/5:
+ commitizen 4.3.1

Done in 7.4s using pnpm v10.27.0
tasavenkova@tasavenkova:~$ pnpm add -g standard-
```

Рисунок 4.2: Установка через pnpm

Создаю новый репозиторий и делаю там первый коммит. (рис. 4.3)

```
/home/tasavenkova/.local/share/pnpm/global/5:
+ standard-changelog 7.0.1

Done in 3.7s using pnpm v10.27.0
tasavenkova@tasavenkova:~$ gh repo create --public git-extended
✓ Created repository tasavenkova/git-extended on github.com
  https://github.com/tasavenkova/git-extended
tasavenkova@tasavenkova:~$ git clone --recursive git@github.com: tasavenkova/git-extended.git
Клонирование в «tasavenkova/git-extended.git»...
fatal: ошибка внешнего репозитория:
  is not a valid repository name
Visit https://support.github.com/ for help
tasavenkova@tasavenkova:~$ git clone --recursive git@github.com:tasavenkova/git-extended.git
Клонирование в «git-extended»...
warning: Похоже, что вы клонировали пустой репозиторий.
tasavenkova@tasavenkova:~$ cd git-extended
tasavenkova@tasavenkova:~/git-extended$ touch README.md && echo "text" > README.md
tasavenkova@tasavenkova:~/git-extended$ git add .
tasavenkova@tasavenkova:~/git-extended$ git commit -m "first commit"
[main (корневой коммит) bfde395] first commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
tasavenkova@tasavenkova:~/git-extended$ git pu
```

Рисунок 4.3: Создание репозитория

Инициализирую и конфигурирую общепринятые коммиты в созданной директории через редактирование package.json. (рис. 4.4)

```
"name": "git-extended",
"version": "1.0.0",
"description": "Git repo for educational purposes",
"main": "index.js",
"repository": "git@github.com:tasavenkova/git-extended.git",
"author": "Tanya Savenkova 1032253537@rudn.ru",
"license": "CC-BY-4.0",
"config": {
  "commitizen": {
    "path": "cz-conventional-changelog"
  }
}
}
```

Рисунок 4.4: Конфигурация package.json

Делаю снимок изменений, создаю коммит и отправляю на удаленный репозиторий. (рис 4.5)

```
tasavenkova@tasavenkova:~/git-extended$ nano package.json
tasavenkova@tasavenkova:~/git-extended$ git add .
tasavenkova@tasavenkova:~/git-extended$ git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: docs:     Documentation only changes
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 94 chars):
 (27) improve information content
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
[main 1e24a9f] docs: improve information content
 1 file changed, 15 insertions(+)
 create mode 100644 package.json
tasavenkova@tasavenkova:~/git-extended$ git push
Перечисление объектов: 4, готово.
Подсчет объектов: 100% (4/4), готово.
При сжатии изменений используется до 5 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (3/3), 1.16 KiB | 1.16 MiB/s, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:tasavenkova/git-extended.git
   bfde395..1e24a9f  main -> main
tasavenkova@tasavenkova:~/git-extended$ git g
```

Рисунок 4.5: Отправление коммита на GitHub

Инициализирую в репозитории git flow и создаю 1 релиз в только что созданной ветке develop. (рис. 4.6)

```
tasavenkova@tasavenkova:~/git-extended$ git push
Перечисление объектов: 4, готово.
Подсчет объектов: 100% (4/4), готово.
При сжатии изменений используется до 5 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (3/3), 1.16 KiB | 1.16 MiB/s, готово.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:tasavenkova/git-extended.git
   bfde395..1e24a9f  main -> main
tasavenkova@tasavenkova:~/git-extended$ git flow init

Which branch should be used for bringing forth production releases?
   - main
Branch name for production releases: [main]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/home/tasavenkova/git-extended/.git/hooks]
tasavenkova@tasavenkova:~/git-extended$ git branch
* develop
  main
tasavenkova@tasavenkova:~/git-extended$ git push
```

Рисунок 4.6: Использование git-flow

Создаю список изменений через standard changelog, заканчиваю релиз и выгружаю на удаленный репозиторий изменения. (рис. 4.7)

```
tasavenkova@tasavenkova:~/git-extended$ git flow release finish 1.0.0
Переключились на ветку «main»
Эта ветка соответствует «origin/main».
Merge made by the 'ort' strategy.
 CHANGELOG.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 CHANGELOG.md
Уже на «main»
Ваша ветка опережает «origin/main» на 2 коммита.
  (используйте «git push», чтобы опубликовать ваши локальные коммиты)
fatal: нет описания метки?
Fatal: Tagging failed. Please run finish again to retry.
tasavenkova@tasavenkova:~/git-extended$ git push --all
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 5 потоков
Сжатие объектов: 100% (3/3), готово.
Запись объектов: 100% (4/4), 1.83 KiB | 1.83 MiB/s, готово.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:tasavenkova/git-extended.git
   1e24a9f..37b8319  main -> main
 * [new branch]      release/1.0.0 -> release/1.0.0
tasavenkova@tasavenkova:~/git-extended$ git push --tags
Everything up-to-date
tasavenkova@tasavenkova:~/git-extended$ gh release create v1.0.0 -F CHANGELOG.md
https://github.com/tasavenkova/git-extended/releases/tag/v1.0.0
tasavenkova@tasavenkova:~/git-extended$ git flow feature
```

Рисунок 4.7: Загрузка релиза на github

Инициализирую ветку feature для работы над новой функциональностью, готовлю релиз и загружаю на github. ([рис.@fig-008])

```
Переключились на новую ветку «release/1.2.3»

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

    git flow release finish '1.2.3'

tasavenkova@tasavenkova:~/git-extended$ nano package.json
tasavenkova@tasavenkova:~/git-extended$ standard-changelog
tasavenkova@tasavenkova:~/git-extended$ git add .
tasavenkova@tasavenkova:~/git-extended$ git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: chore:    Other changes that don't modify src or test files
? What is the scope of this change (e.g. component or file name): (press enter to skip) site
? Write a short, imperative tense description of the change (max 87 chars):
 (16) update changelog
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
[release/1.2.3 a926be1] chore(site): update changelog
 2 files changed, 8 insertions(+), 1 deletion(-)
```

Рисунок 4.8: Работа с новой веткой и загрузка релиза

5 Выводы

В ходе выполнения лабораторный работы я получила навыки правильной работы с репозиториями git

Список литературы